

Lab B - 자동화 파이프라인

이슈에서 PR까지, 사람 없이 도는 파이프라인

One agent for everywhere you code - CLI, IDE, App, Web

Codex Deep Dive Workshop (2-Day) - Module 10 / Lab B

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



MODULE 10

Lab B - 자동화 파이프라인

이슈에서 PR까지, 사람 없이 도는 파이프라인



랩 개요와 사전 준비

무엇을 만들고 무엇이 필요한가

1 Local

exec 로컬 검증

2

Workflow

워크플로우 작성

3 Trigger

라벨 트리거

4 Verify

검증 게이트

5 Deliver

PR, 알림

6 Secure

안전장치 점검

PREREQUISITES

사전 준비물

- Lab A에서 사용한 본인 레포 (테스트와 린트 명령이 있는 프로젝트)
- OpenAI API 키, GitHub Secrets 등록 권한
- GitHub Actions가 활성화된 리포, gh CLI 권장
- Module 9의 워크플로우 예시 (S8 GitHub Action 슬라이드)

BUILD ON LAB A Lab A에서 PR을 만든 그 레포를 이어서 씁니다.

수동 작업을 오늘 자동 파이프라인으로 바꾸는 것이 목표입니다.

6 Steps

파이프라인 구축

80 min

안전 점검 포함

본인 레포

Lab A 연속

Secrets

안전한 키 주입

Step 1. exec 로컬 검증 (10분)

자동화 전에 손으로 확인

Terminal - Step 1

```
# 1-1. 자동화할 작업을 exec로 먼저 실행 (기본은 read-only)
$ codex exec "Run lint and tests, fix violations"
$ echo $? # 0=성공, 비0=실패
# 1-2. JSON Lines 이벤트 스트림으로 실행 (CI 파싱용)
$ codex exec --json "Run lint and tests" > events.jsonl
# 1-3. 최종 메시지만 파일로 (구조화가 필요하면 --output-schema)
$ codex exec -o result.txt "Summarize the fix"
```

CHECKPOINT exec가 종료 코드 0을 반환하고 events.jsonl의 마지막 agent_message를 확인하면 통과입니다.
로컬에서 안 되는 작업은 CI에서도 안 됩니다.

10 min

Step 1 시간

\$?

종료 코드 확인

--json

이벤트 스트림

Local first

CI 전 손 검증

Step 2. 워크플로우 작성 (15분)

트리거와 실행 정의

● ● ● .github/workflows/codex-fix.yml

```
name: Codex Auto Fix
on: { issues: { types: [labeled] } }
permissions: { contents: write, pull-requests: write }
jobs:
  fix:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: openai/codex-action@v1
        with: { openai-api-key: ${{ secrets.OPENAI_API_KEY }}, prompt: "Resolve the labeled issue" }
```

CHECKPOINT 입력은 task가 아니라 prompt(또는 prompt-file)와 openai-api-key가 핵심입니다.
파일을 푸시한 뒤 Actions 탭에 워크플로우가 나타나면 통과입니다.

15 min

Step 2 시간

on:

트리거 정의

checkout

코드 가져오기

action

에이전트 실행

Step 3. 시크릿과 트리거 (10분)

안전한 키 주입과 첫 실행

Terminal - Step 3

```
# 3-1. API 키를 GitHub Secrets로 등록 (gh CLI)
$ gh secret set OPENAI_API_KEY

# 3-2. 워크플로우 푸시
$ git add -A && git commit -m "ci: codex auto fix" && git push

# 3-3. 이슈에 라벨을 붙여 트리거
GitHub UI에서 이슈 생성 후 지정 라벨 부착
```

보안 / CHECKPOINT API 키는 절대 코드나 with에 직접 쓰지 말고 GitHub Secrets로만 주입하십시오.
시크릿 등록, 푸시 후 워크플로우 노출, 라벨 트리거 동작을 확인하면 통과입니다.

10 min

Step 3 시간

gh secret

키 안전 등록

push

워크플로우 배포

label

라벨로 트리거

Step 4. 검증 게이트 (15분)

에이전트 변경도 동일하게 검증

ADD GATES

검증 단계 추가

- 워크플로우에 테스트 실행 step 추가
- 린트와 정적 분석을 게이트로 연결
- 실패 시 워크플로우를 중단하도록 설정
- PR에는 보호 브랜치 규칙 적용

WHY

왜 필요한가

- 사람 감독이 없으므로 자동 검증이 안전함
- 에이전트 변경도 사람 변경과 동일 기준
- 검증 통과 전에는 머지 불가
- 실패한 변경이 메인에 들어가는 것을 차단

CHECKPOINT 테스트와 린트 게이트가 워크플로우에 연결되고, 일부러 실패하는 변경이 게이트에서 막히는 것을 확인하면 통과입니다.

15 min

Step 4 시간

Test

테스트 게이트

Lint

정적 분석

Block

실패 시 차단

Step 5. PR과 알림 (15분)

결과를 사람에게 전달

1 PR 생성

변경을 PR로

2 요약 첨부

본문에 설명

3 알림

Slack 통지

4 리뷰 대기

사람 결정

DELIVER

전달 단계 구성

- 에이전트가 PR을 생성하고 본문에 작업 요약과 검증 결과 포함
- Slack 연동으로 PR 링크를 팀 채널에 알림 (Module 8 Slack 통합 활용)
- 사람은 알림을 받아 PR을 리뷰하고 머지 여부를 결정, 자동화는 PR까지만

CHECKPOINT PR이 자동 생성되고 요약이 포함되며, 알림이 도착하면 통과입니다. 머지 결정권은 끝까지 사람에게 있어야 합니다.

15 min

Step 5 시간

PR

자동 생성

Notify

Slack 알림

사람

머지 결정

가드레일과 안전한 트리거

무인 실행의 비용/보안 방어선

● ● ● .github/workflows/codex-fix.yml - guardrails

```
# 무인 자동화의 방어선은 가드레일 (Step 6와 연결)
permissions: { contents: write, pull-requests: write } # 최소 권한
concurrency: # 중복 실행 취소
  group: codex-`${{ github.event.issue.number }}`
  cancel-in-progress: true
jobs:
  fix:
    runs-on: ubuntu-latest
    timeout-minutes: 15 # 폭주 비용 차단
    steps:
      - uses: openai/codex-action@v1
        with: { openai-api-key: `${{ secrets.OPENAI_API_KEY }}`, safety-strategy: drop-sudo }
```

GUARDRAILS 무인 실행은 가드레일이 유일한 방어선입니다.

키는 env가 아닌 action input으로 주입하고, PR과 이슈 본문의 프롬프트 인젝션을 점검하며, Codex 단계를 잡의 마지막에 두십시오.

최소 권한

토큰 범위 제한

timeout

비용 폭주 차단

input 주입

env 노출 금지

concurrency

중복 실행 취소

Step 6. 안전장치 점검 (15분)

자동화의 가드레일 확인

01	격리 환경	CI 러너가 일회용 격리 환경인지 확인	필수
02	최소 권한	토큰과 접근 범위를 최소로 제한했는지	필수
03	시크릿 격리	키가 Secrets로만 주입되고 로그에 없는지	필수
04	검증 게이트	테스트와 보호 브랜치가 작동하는지	필수
05	감사 로그	실행 로그가 보존되어 추적 가능한지	권장

ASSUME BREACH 자동화는 사람이 매번 보지 않으므로 가드레일이 유일한 방어선입니다. 다섯 항목을 모두 점검하면 통과입니다.

15 min

Step 6 시간

5 체크

가드레일 항목

격리

일회용 러너

최소 권한

범위 제한

파이프라인 전체 점검

트리거에서 전달까지

1 Trigger

이슈 라벨

2 Execute

격리 실행

3 Verify

테스트 게이트

4 Deliver

PR, 알림

FULL CIRCLE Module 9의 레퍼런스 아키텍처를 직접 구현했습니다.
이슈 라벨 하나가 격리 실행, 자동 검증, PR 생성, 알림까지 사람 없이 이어집니다.

TRIGGER

이벤트 시작

이슈 라벨이 파이프라인을 깨웁니다

GATE

안전 검증

테스트와 보호 브랜치가 품질을 지킵니다

HUMAN

최종 결정

PR 리뷰와 머지는 사람의 몫입니다

4 Stages

전체 파이프라인

1 Label

단일 트리거

Auto

검증까지 자동

Human

머지는 사람

랩 체크포인트 총정리

여섯 단계 최종 점검

1	로컬 검증	exec가 종료 코드 0과 JSON 출력 반환	Step 1
2	워크플로우	파일 생성 후 Actions 탭에 노출	Step 2
3	트리거	시크릿 등록과 라벨 트리거 동작	Step 3
4	검증 게이트	실패 변경이 게이트에서 차단됨	Step 4
5	PR과 알림	요약 포함 PR 생성과 알림 도착	Step 5
6	안전장치	격리, 최소 권한, 시크릿 격리 점검	Step 6

6/6	1 파이프라인	Trigger to PR	Day 2
완료 체크포인트	팀 자동화 자산	전 과정 자동	종합 실습 완주

막혔을 때

자동화 트러블슈팅 가이드

01	워크플로우 안 뜸	yml 문법 확인, .github/workflows 경로 점검	설정
02	트리거 안 됨	on 절 조건과 라벨 이름 일치 확인	트리거
03	인증 실패	Secrets 등록 확인, 키 이름 일치 점검	인증
04	그래도 안 됨	Actions 로그 확인 후 진행자 호출	지원

LOGS FIRST 자동화 문제의 첫 단서는 GitHub Actions 실행 로그입니다. 로그를 먼저 펼쳐 어느 step에서 멈췄는지 확인하십시오.

yml
문법 1차 점검

on:
트리거 조건

Secrets
인증 점검

Logs
실행 로그 확인

Lab B 마무리

수동에서 자동으로

WELL DONE

수고하셨습니다. Lab A의 수동 작업을 Lab B에서 자동 파이프라인으로 바꿨습니다.
이슈 라벨 하나가 검증된 PR로 이어지는 흐름을 직접 만들었습니다.

BUILT

파이프라인

트리거에서 전달까지 자동 흐름을 구축했습니다

LEARNED

자동화 설계

Module 9의 아키텍처를 직접 구현했습니다

PRINCIPLE

핵심 원칙

가드레일과 사람의 최종 결정 권입니다

NEXT

다음 예고

Enterprise 도입과 Best Practices로 마무리합니다

Trigger to PR

자동 파이프라인

Gate

검증 게이트

Human

머지 결정

Next

Enterprise

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

